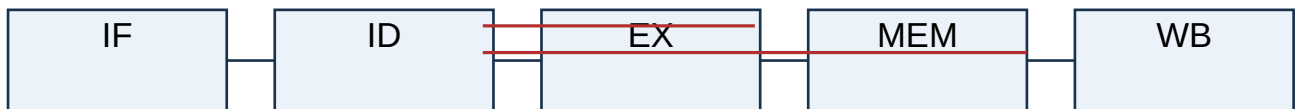


Lab 06 Report: Forwarding and Branch Prediction

Pipelined MIPS-like CPU implementation

This report describes the HW06 implementation built from the previous pipelined CPU. The goal was to preserve functional correctness while reducing unnecessary stalls with forwarding and adding a direct-mapped BTB plus a 2-bit single-level pattern predictor. The submitted design keeps the original five-stage structure and adds small, explicit modules for the new behavior.



Forward paths: EX/MEM -> EX, MEM/WB -> EX, MEM/WB -> RF reads

The design still uses IF, ID, EX, MEM, and WB pipeline registers. Fetch now obtains a predicted next PC from BP.v, and the predicted PC is carried through IF/ID and ID/EX so the branch resolution stage can compare it with the actual next PC. Data forwarding is handled by FORWARD.v and is applied to both ALU operands and store write data before EX/MEM latching.

Changed or added files: CPU.v integrates forwarding and prediction, HAZARD.v implements the reduced stall policy, FORWARD.v selects EX operands, BP.v implements BTB/PHT state, RF.v retains internal write-to-read forwarding, and Makefile includes the added modules.

Data Forwarding

Forwarding cases implemented

The forwarding unit compares ID/EX source registers against EX/MEM and MEM/WB destination registers. EX/MEM forwarding is used for ALU and JAL results that are already available before the next EX stage. MEM/WB forwarding is used for results two cycles older, including loaded memory data.

ALU operand forwarding

CPU.v creates forwarded versions of rd1 and rd2 before selecting ALU operands. For R-type operations, branches, and address calculations, the latest available producer wins. EX/MEM has priority over MEM/WB because it is the newer value.

Store-data forwarding

The value latched into EX/MEM.rd2 is the forwarded rt value, not the raw ID/EX rd2. This lets an instruction such as addiu followed by sw store the just-produced value without waiting for register writeback.

Internal RF forwarding

RF.v already returns wr_data when a read address matches the WB write address in the same cycle. This covers the $\text{dist}(i,j)=3$ case required in the handout and avoids an avoidable WB-to-ID stall.

Hazard Detection and Stall Policy

Reduced RAW stalls

The previous HW05 unit stalled on many register matches. HW06 stalls only where forwarding cannot supply the value in time. The main remaining case is load-use when the load is in EX and the consumer is in ID.

Operand awareness

The hazard unit receives `uses_rs` and `uses_rt` from CPU.v. This prevents false stalls for immediates and jump target fields that happen to share bit positions with register fields.

JR exception

JR is resolved in ID and is not supported by the branch predictor as required. Because normal EX forwarding cannot help the ID-stage target calculation, the hazard unit stalls JR when its source is still being produced in EX or is an unresolved load in MEM.

Pipeline bubbles

On a true stall, PC and IF/ID are held while ID/EX is cleared into a bubble. On a branch misprediction, IF/ID and ID/EX are flushed so wrong-path instructions cannot commit architectural state.

Branch Prediction

BTB structure

BP.v implements a 64-entry direct-mapped BTB. The index is PC[7:2], and the tag is PC[31:8], giving the requested 6-bit index and 24-bit tag. Each entry stores a two-bit type field: empty, jump, or branch.

PHT structure

The pattern history table has 256 entries addressed by PC[9:2]. Each entry is a 2-bit saturating counter. The most significant bit is used as the taken prediction for BTB branch entries.

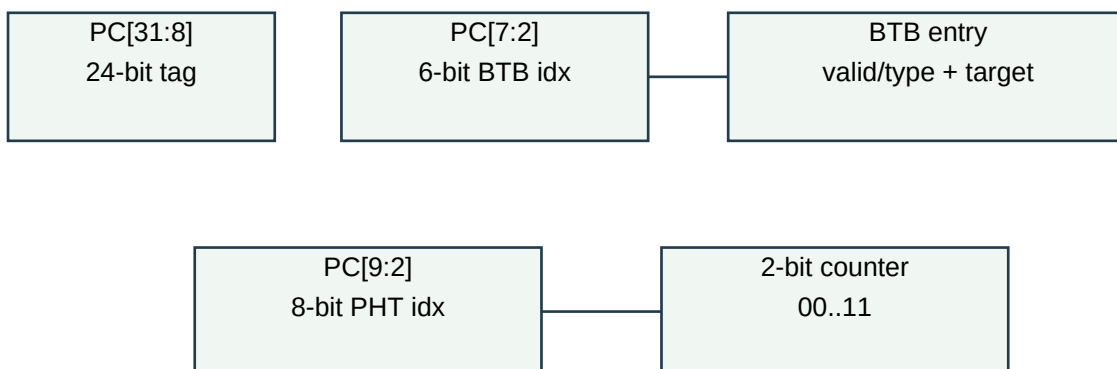
Prediction action

On a BTB hit, jump entries predict taken to the stored target. Branch entries predict to the stored target only when the PHT counter is in a taken state. Otherwise fetch proceeds with PC+4.

Update timing

Branches update the BTB/PHT in EX when the ALU comparison resolves. J and JAL update the BTB in ID. JR is deliberately excluded from BP support, matching the assignment statement.

Branch predictor organization



Correctness and Verification

Existing regression tests

The HW05 testcase1 through testcase8 programs were copied into HW06 and run with the new Verilog files. Each case compiled with iverilog and finished with Simulation success!!! under CPU_tb.v.

Functional compatibility

The new predictor changes only fetch direction and flush behavior. Register and memory state are still checked by the original testbench against reference_reg.mem and reference_mem.mem, so wrong-path writes or missed forwarding would be caught by the final architectural comparison.

Performance intent

Forwarding removes ALU-to-ALU, ALU-to-store, and MEM/WB-to-EX stalls. The predictor reduces control penalties for repeated branches and direct jumps once BTB/PHT entries have been trained.

Known scope

The implementation follows the assignment scope: direct-mapped BTB, 8-bit PHT index, 2-bit saturating counters, and no JR prediction. The testbench verifies correctness, not CPI measurement.

Implementation Notes

Module-level summary

CPU.v: Adds predicted PC handling, IF/ID and ID/EX prediction metadata, EX-stage branch misprediction recovery, ID-stage direct jump recovery, forwarded ALU operands, and forwarded store data. The PC update priority is reset, branch correction, jump correction, predicted fetch when not stalled.

FORWARD.v: Produces two 2-bit select signals. 00 selects the original ID/EX register value, 10 selects the EX/MEM result, and 01 selects the MEM/WB writeback data. EX/MEM load forwarding is blocked because load data is not available until MEM/WB.

BP.v: Initializes BTB entries as empty and PHT counters as weakly not taken. On update, branch entries write the branch target and train the saturating counter. Jump entries write the direct jump target and are always predicted taken on future BTB hits.

HAZARD.v: Detects true load-use stalls and the special JR source hazards. It no longer stalls for ordinary ALU dependencies because those are handled by the forwarding network. This satisfies the forwarding requirement while retaining correctness for values that cannot physically be forwarded in time.